# SI405 – Análise de Sistemas de Informação II

# Princípios SOLID – PART II

Prof. Ricardo Vilela

rfvilela@unicamp.br

# RECAPITULANDO...

## ➢O que é SOLID?

**SOLID** é um conjunto de **cinco princípios de design orientado a objetos** que ajudam a construir sistemas de software:

- mais **organizados**
- mais **flexíveis**
- mais **fáceis de manter**
- mais **fáceis de evoluir**

# RECAPITULANDO...

- O que significa SOLID?

- Acrônimo formado pelas iniciais de cinco princípios de design
  - **S**ingle Responsability Principle
  - **O**pen/Closed Principle
  - **L**iskov Substitution Principle
  - **I**nterface Segregation Principle
  - **D**ependency Inversion Principle

**Importante:**

SOLID **não é uma regra rígida**, mas um **conjunto de boas práticas** que ajudam a tomar decisões de design melhores.

# COESÃO DE INTERFACES

- Vimos anteriormente que coesão é fundamental para manutenção e reuso de nosso código.
- Mas até então, discutimos coesão em nível de classe.
- Mas e Interfaces?

# COESÃO DE INTERFACES

- Vimos anteriormente que coesão é fundamental para manutenção e reuso de nosso código.
- Mas até então, discutimos coesão em nível de classe.
- Mas e Interfaces? Elas precisam ser Coesas?

# COESÃO DE INTERFACES

- O que significa uma Interface coesa?

# COESÃO DE INTERFACES

- **Definindo interfaces coesas:**
  - Dilema comum: Qual parâmetro devo receber meu método?
  - Exemplo: Considere a classe ***NotaFiscal*** que é extremamente complexa:

```
class NotaFiscal {
    public NotaFiscal(
        Cliente cliente,
        List<Item> itens,
        List<Desconto> descontos,
        Endereco entrega,
        Endereco cobranca,
        FormaDePagamento pagto,
        double valorTotal
    ) { ...}

    // muitos atributos e métodos
}
```

# COESÃO DE INTERFACES

- **Definindo interfaces coesas:**
  - Em seguida, considere um método responsável por calcular o imposto dessa nota fiscal.

```
class CalculadorDeImposto {
    public double calcula(NotaFiscal nf) {
        double total = 0;
        for(Item item : nf.getItens()) {
            if(item.getValor()>1000)
                total+= item.getValor() * 0.02;
            else
                total+= item.getValor() * 0.01;
        }
        return total;
    }
}
```

# COESÃO DE INTERFACES

- **Definindo interfaces coesas:**
  - Veja que o método calcula() recebe uma NotaFiscal de forma desnecessária.
  - Seria possível receber apenas uma List<Item>.

```
class CalculadorDeImposto {
    public double calcula(NotaFiscal nf) {
        double total = 0;
        for(Item item : nf.getItens()) {
            if(item.getValor()>1000)
                total+= item.getValor() * 0.02;
            else
                total+= item.getValor() * 0.01;
        }
        return total;
    }
}
```

# COESÃO DE INTERFACES

- Definindo interfaces coesas:
  - Dilema: *Qual dos dois devemos receber?*

```
class CalculadorDeImposto {
    public double calcula(NotaFiscal nf) {
        double total = 0;
        for(Item item : nf.getItens()) {
            if(item.getValor()>1000)
                total+= item.getValor() * 0.02;
            else
                total+= item.getValor() * 0.01;
        }
        return total;
    }
}
```

# PRINCÍPIO DE SEGREGAÇÃO DE INTERFACES (ISP)

- **Definindo interfaces coesas:**
  - **Ideia principal:**
    - Preferir dependências **mais simples e estáveis**
    - Evitar acoplamento com classes **complexas e voláteis**
  - **Problema:**
    - NotaFiscal é complexa
    - método depende de algo instável
    - maior chance de mudanças e quebra

```
class CalculadorDeImposto {
    public double calcula(NotaFiscal nf) {
        double total = 0;
        for(Item item : nf.getItens()) {
            if(item.getValor()>1000)
                total+= item.getValor() * 0.02;
            else
                total+= item.getValor() * 0.01;
        }
        return total;
    }
}
```

# COESÃO DE INTERFACES

- **Definindo interfaces coesas:**
  - **Tentativa de simplificação:**
    - Parâmetro apenas com lista de itens
    - Ou double total
    - Código mais simples

```
class CalculadorDeImposto {
    public double calcula(NotaFiscal nf) {
        double total = 0;
        for(Item item : nf.getItens()) {
            if(item.getValor()>1000)
                total+= item.getValor() * 0.02;
            else
                total+= item.getValor() * 0.01;
        }
        return total;
    }
}
```

# COESÃO DE INTERFACES

- Definindo interfaces coesas:
  - Tentativa de simplificação:
    - Parâmetro apenas com lista de itens
    - Ou double total
    - Código mais simples

```
class CalculadorDeImposto {
    public double calcula(NotaFiscal nf) {
        double total = 0;
        for(Item item : nf.getItens()) {
            if(item.getValor()>1000)
                total+= item.getValor() * 0.02;
            else
                total+= item.getValor() * 0.01;
        }
        return total;
    }
}
```

**PERDE SIGNIFICADO E CONTEXTO**

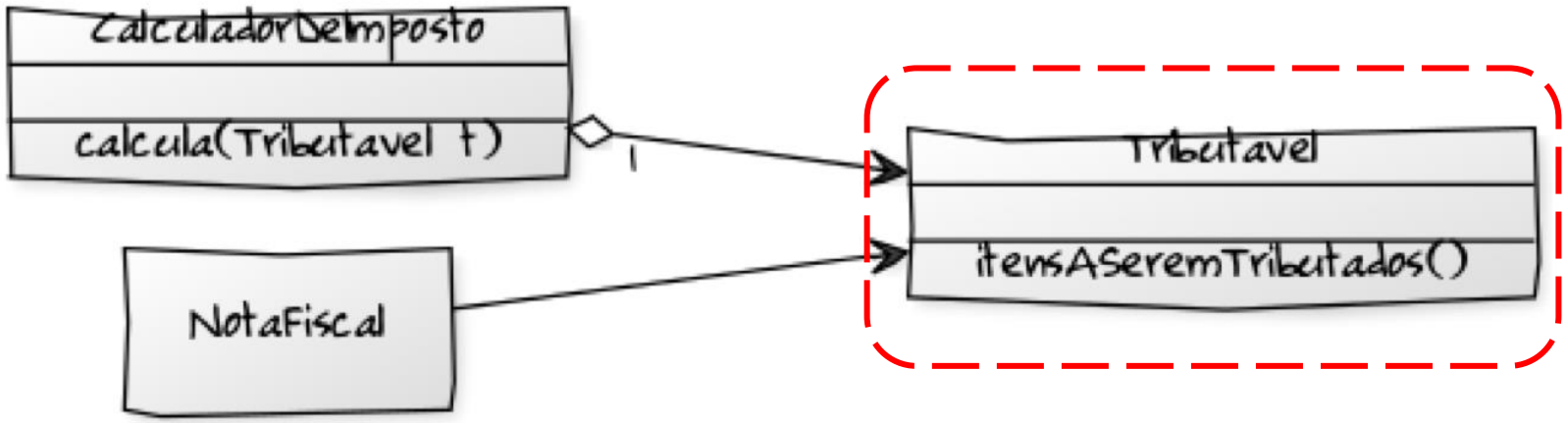
# COESÃO DE INTERFACES

- **Definindo interfaces coesas:**
  - **Novo problema:**
    - Parâmetros Genéricos demais
    - Não deixam claro **o que o método realmente precisa**

```
class CalculadorDeImposto {
    public double calcula(NotaFiscal nf) {
        double total = 0;
        for(Item item : nf.getItens()) {
            if(item.getValor()>1000)
                total+= item.getValor() * 0.02;
            else
                total+= item.getValor() * 0.01;
        }
        return total;
    }
}
```

# COESÃO DE INTERFACES

- Definindo interfaces coesas:
  - Solução elegante:




Criação da interface *Tributavel*

# COESÃO DE INTERFACES

- **Definindo interfaces coesas:**

```
interface Tributavel {
    List<Item> itensASeremTributados();
}
```

```
class NotaFiscal implements Tributavel {
    public NotaFiscal(...) { ... }

    public List<Item> itensASeremTributados() { ... }
}
```

```
class CalculadorDeImposto {
    public double calcula(Tributavel t) {
        double total = 0;
        for(Item item : t.itensASeremTributados()) {
            if(item.getValor()>1000)
                total+= item.getValor() * 0.02;
            else
                total+= item.getValor() * 0.01;
        }
        return total;
    }
}
```

# COESÃO DE INTERFACES

- **Definindo interfaces coesas:**
  - Diminuímos o risco do acoplamento, pois a interface ***Tributavel*** é muito mais estável que a classe *NotaFiscal*.
  - O método calcula() recebe agora apenas o que realmente precisa.
    - Tributavel é uma interface segura
  - Temos semântica no parâmetro do método. Não recebemos uma lista de double ou um double qualquer, e sim algo passível de tributação.

# PRINCÍPIO DE SEGREGAÇÃO DE INTERFACES (ISP)

# PRINCÍPIO DE SEGREGAÇÃO DE INTERFACES (ISP)

- Veja a interface a seguir, responsável por calcular um imposto e gerar uma NotaFiscal:

```
interface Imposto {
    NotaFiscal geraNota();
    double imposto(double valorCheio);
}
```

# PRINCÍPIO DE SEGREGAÇÃO DE INTERFACES (ISP)

- Imagine agora uma primeira implementação dessa calculadora para o imposto ISS:

```
class ISS implements Imposto {
    public double imposto(double valorCheio) {
        return 0.1 * valorCheio;
    }

    public NotaFiscal geraNota() {
        return new NotaFiscal(
            "Alguma informacao aqui",
            "Alguma outra informacao aqui"
        );
    }
}
```

# PRINCÍPIO DE SEGREGAÇÃO DE INTERFACES (ISP)

- Agora imagine um novo imposto, chamado IXMX, que é calculado também sobre o valor cheio, mas não emite nota fiscal:

```
class IXMX implements Imposto {
    public double imposto(double valorCheio) {
        return 0.2 * valorCheio;
    }

    public NotaFiscal geraNota() {
        // lanca uma exceção
        throw new NaoGeraNotaException();
        // ou retornar nulo
        return null;
    }
}
```

# PRINCÍPIO DE SEGREGAÇÃO DE INTERFACES (ISP)

- Interfaces com muitas responsabilidades podem violar o design correto.
- Alterações nessas interfaces podem quebrar as classes clientes.
- Isso ocorre porque mudam o contrato da interface
- Consequências:
  - Alto acoplamento
  - Maior risco de defeitos no sistema
  - Dificuldade de manutenção

# PRINCÍPIO DE SEGREGAÇÃO DE INTERFACES (ISP)

- A solução para o problema é análoga ao que tomamos quando discutimos classes coesas:

```
interface CalculadorDeImposto {
    double imposto(double valorCheio);
}

interface GeradorDeNota {
    NotaFical geraNota();
}
```

# PRINCÍPIO DE SEGREGAÇÃO DE INTERFACES (ISP)

- Dessa forma, a classe filha implementa quais interfaces forem necessárias, sem precisar fazer “gambiarras” para adequar uma interface que não faz sentido para ela:

```
class ISS implements CalculadorDeImposto, GeradorDeNota {
    // os dois métodos aqui
}

class IXMX implements CalculadorDeImposto {
    // só implemementa uma interface, pois
    // esse aqui não gera nota fiscal
}
```

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

# PRINCÍPIO DE SEGREGAÇÃO DE INTERFACES (ISP)

- Módulos de alto nível não devem depender de módulos de baixo nível.
- Ambos devem depender de abstrações.

**Ideia central:**
Depender de interfaces, não de implementações.

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- Ao invés de:
  - Classe A -> depende de Classe B
- Devemos ter:
  - Classe A -> depende de Interface
  - Classe B - > implementa interface

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **Entendendo o problema:**

```
1   public class PedidoService {
2
3       private EmailService emailService = new EmailService();
4
5       public void processar(Pedido pedido) {
6           emailService.enviar(pedido);
7       }
8   }
```

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **Entendendo o problema:**
  - Classe de alto nível (*PedidoService*) depende de classe de baixo nível (*EmailService*)

```
1   public class PedidoService {
2
3       private EmailService emailService = new EmailService();
4
5       public void processar(Pedido pedido) {
6           emailService.enviar(pedido);
7       }
8   }
```

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **Entendendo o problema:**
  - Consequências:
    - Alto acoplamento
    - Dificulta eventual manutenção
    - Difícil de testar
    - Código pouco flexível

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **Ideia de Solução:**
  - Inversão de Dependência:
    - Antes:
      - PedidoService -> EmailService
    - Depois:
      - PedidoService -> Notificador <- EmailService

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **Ideia de Solução:**
  - Inversão de Dependência:

```java
public interface Notificador {

    void enviar(Pedido pedido);

}
```

```java
public class EmailService implements Notificador {

    public void enviar(Pedido pedido) {
        System.out.println(x: "Enviando email...");
    }
}
```

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **Ideia de Solução:**
  - Aplicando Inversão:
    - Antes:
      - ***PedidoService*** depende de ***EmailService***
    - *Depois:*
      - ***PedidoService*** *depende de* ***Notificador***

```java
public class PedidoService {

    private Notificador notificador;

    public PedidoService(Notificador notificador) {
        this.notificador = notificador;
    }

    public void processar(Pedido pedido) {
        notificador.enviar(pedido);
    }
}
```

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **Ideia de Solução:**
  - Aplicando Inversão:
    - Antes:
      - ***PedidoService*** depende de ***EmailService***
    - *Depois:*
      - ***PedidoService*** *depende de* ***Notificador***

INJEÇÃO DE DEPENDÊNCIA

```
1   public class PedidoService {
2
3       private Notificador notificador;
4
5       public PedidoService(Notificador notificador) {
6           this.notificador = notificador;
7       }
8
9       public void processar(Pedido pedido) {
10          notificador.enviar(pedido);
11      }
12  }
```

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **Possível Modificação:**

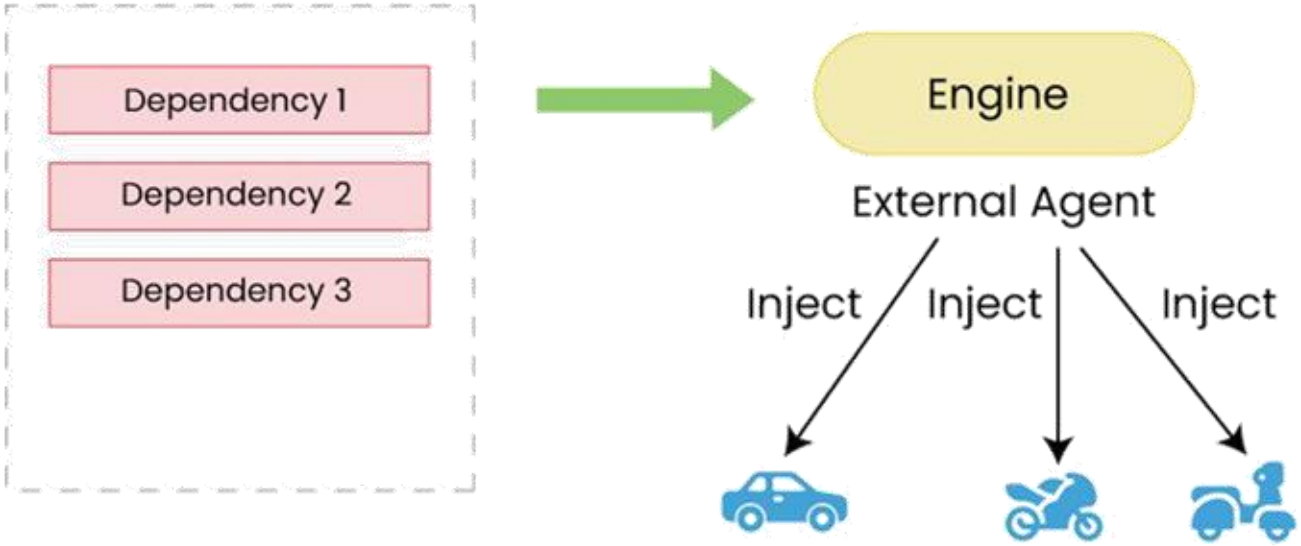
```
1   public class SmsService implements Notificador {
2
3       public void enviar(Pedido pedido) {
4           System.out.println(x: "Enviando SMS...");
5       }
6   }
```

- Não resulta em modificação da classe PedidoService

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

## ❑INJEÇÃO DE DEPENDÊNCIA:

❑Técnica em que uma classe recebe suas dependências em vez de cria-las.



# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **INJEÇÃO DE DEPENDÊNCIA:**
  - Técnica em que uma classe recebe suas dependências em vez de cria-las.

```
PedidoService notificador = new EmailService();
PedidoService service = new PedidoService(notificador);
```

# PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIA (DIP)

- **Resumo do DIP:**
  - Módulos de alto nível não devem depender de detalhes
  - Ambos devem depender de abstrações

# SI405 – Análise de Sistemas de Informação II

# Princípios SOLID – PART II

Prof. Ricardo Vilela

rfvilela@unicamp.br